

AULA 5

RVIZ & URDF

CONTINUAÇÃO DA AULA PASSADA: CÓDIGO DO PACOTE TF2_DEMO

- Pacote tf2_demo está disponível na pasta Aula4 no repositório https://github.com/nycolasRamires/ros2_ufc/
- Coloque a pasta tf2_demo no diretório src do seu workspace
- Instale o pacote com o seguinte comando no diretório do workspace:

```
colcon build -symlink-install && source  
install/setup.bash
```
- Lance o pacote com
 - `ros2 launch tf2_demo demo_bringup.launch.xml config:=True`
- Contém três executáveis:
 - `drone_tf_pub` – publica a pose do drone, variando no tempo
 - `static_cargo_pub` – publica um frame estático, fixo ao drone
 - `cargo_pubsub` – escuta e publica o frame dinâmico de um payload com relação ao drone

Estrutura do pacote:

```
tf2_demo/  
├── CMakeLists.txt  
├── include  
│   └── tf2_demo  
│       ├── cargo_pubsub.hpp  
│       └── drone_tf_pub.hpp  
├── launch  
│   ├── demo_bringup.launch.xml  
│   └── static_frames.launch.xml  
├── LICENSE  
├── package.xml  
├── rviz  
│   └── frame_demo.rviz  
└── src  
    ├── cargo_pubsub.cpp  
    ├── drone_tf_pub.cpp  
    └── static_drone_pub.cpp
```

CÓDIGO DO PACOTE TF2_DEMO

drone_tf_pub: Publica frame drone_frame com relação a um frame center

- Declaração de classe em drone_tf_pub.hpp. Na construção:
 - Declara parâmetros speed e drone_tf_name
 - Inicializa tf2_ros::TransformBroadcaster
 - Inicializa rclcpp::TimerBase com função de call-back timer_cb()
 - Salva o tempo de inicialização -> será usado para calcular o quanto que o frame se moveu

```
class DroneTFPublisher : public rclcpp::Node{
public:
    DroneTFPublisher() : Node("drone_pub")
    {
        this->declare_parameter<double>("speed", 1.0); //velocidade do frame
        this->declare_parameter<std::string>("drone_tf_name", "drone_frame");
        drone_frame_ = this->get_parameter("drone_tf_name").as_string();

        // Inicializa o broadcaster de transformação
        tf_broadcaster_ =
            std::make_unique<tf2_ros::TransformBroadcaster>(*this);

        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(50),
            std::bind(&DroneTFPublisher::timer_cb, this)
        );
        last_time_ = this->get_clock()->now().seconds();
    }

private:
    std::unique_ptr<tf2_ros::TransformBroadcaster> tf_broadcaster_;
    rclcpp::TimerBase::SharedPtr timer_;

    void timer_cb();
    void viviani_calculator_(const double theta);

    double theta_;
    double last_time_=0.0;

    double speed_;
    std::string drone_frame_="drone_frame";
};
```

CÓDIGO DO PACOTE TF2_DEMO

drone_tf_pub: Função timer_cb_()

- Calcula o valor de theta (posição atual do drone na curva paramétrica) e chama viviani_calculator_(theta)
- viviani_calculator_ calcula e publica o frame

```
void DroneTFPublisher::timer_cb(){  
    // Pega tempo decorrido entre o último callback e atual  
    auto current_time_ = this->get_clock()->now().seconds();  
    double dt = current_time_ - last_time_;  
    last_time_ = current_time_;  
  
    // Atualiza o parâmetro de comprimento de arco do frame  
    theta_ = this->get_parameter("speed").as_double()*dt + theta_;  
    viviani_calculator_(theta_);  
}
```

CÓDIGO DO PACOTE TF2_DEMO

drone_tf_pub: viviani_calculator_()

- Calcula posição e orientação do frame
 - Orientação é calculada como uma matriz 3x3 -> transformada para quaternion
 - A rigor orientação poderia ser calculada de outra forma
- Monta uma mensagem TransformStamped e publica com o tf_broadcaster_

```
void DroneTFPublisher::viviani_calculator_(const double th){  
    // Posição do frame do drone  
    double x = sin(th)*cos(th);  
    double y = sin(th)*sin(th);  
    double z = cos(th);  
  
    // Operações para definir orientação do drone:  
    //      Frame de Frenet-Serret (Detalhes no código)  
    Eigen::Vector3d T = ...; //Tangente (Eixo X)  
    Eigen::Vector3d N = ...; //Normal (Eixo Y)  
    Eigen::Vector3d B = ...; //Binormal (Eixo Z)  
  
    // Monta o frame de Frenet-Serret  
    tf2::Matrix3x3 fs_frame(  
        T.x(), N.x(), B.x(),  
        T.y(), N.y(), B.y(),  
        T.z(), N.z(), B.z() );  
  
    // Passa para quaternion  
    tf2::Quaternion q;  
    fs_frame.getRotation(q);  
    q.normalize();  
  
    // Cria e preenche a mensagem de transformação  
    geometry_msgs::msg::TransformStamped viviani_tf;  
    viviani_tf.header.stamp = this->get_clock()->now();  
    viviani_tf.header.frame_id = "center";  
    viviani_tf.child_frame_id = drone_frame_;  
  
    viviani_tf.transform.translation.x = x;  
    viviani_tf.transform.translation.y = y;  
    viviani_tf.transform.translation.z = z;  
  
    viviani_tf.transform.rotation = tf2::toMsg(q);  
  
    tf_broadcaster_->sendTransform(viviani_tf);  
}
```

CÓDIGO DO PACOTE TF2_DEMO

static_cargo_pub: Publica uma transformação fixa com relação ao frame do drone

- Classe implementa um nó cujo processamento ocorre uma só vez, na construção
- main serve só para interpretar os argumentos de linha de comando
- Funcionalidade reduzida comparada ao static_transform_Publisher do pacote tf2_ros

```
class CargoFramePublisher : public rclcpp::Node
{
public:
    explicit CargoFramePublisher(char * transformation[], std::vector<double> xyz_vals)
        : Node("static_cargo_tf2_broadcaster")
    {
        tf_static_broadcaster_ =
            std::make_shared<tf2_ros::StaticTransformBroadcaster>(this);

        this->make_transform(transformation, xyz_vals);
    }

private:
    void make_transform(char * transformation[], std::vector<double> xyz_vals)
    {
        geometry_msgs::msg::TransformStamped t;

        t.header.stamp = this->get_clock()->now();
        t.header.frame_id = transformation[1]; // frame do drone
        t.child_frame_id = transformation[2]; // frame da carga

        t.transform.translation.x = xyz_vals[0];
        t.transform.translation.y = xyz_vals[1];
        t.transform.translation.z = xyz_vals[2];
        tf2::Quaternion q;
        q.setRPY(0.0,0.0,0.0);
        t.transform.rotation = tf2::toMsg(q);
        t.transform.rotation.x = q.x();
        t.transform.rotation.y = q.y();
        t.transform.rotation.z = q.z();
        t.transform.rotation.w = q.w();

        tf_static_broadcaster_->sendTransform(t);
    }

    std::shared_ptr<tf2_ros::StaticTransformBroadcaster> tf_static_broadcaster_;
};
```

CÓDIGO DO PACOTE TF2_DEMO

cargo_pubsub: Transformação dinâmica do drone para o payload

- Construtor da classe contém TransformBroadcaster, TransformListener e Buffer
- TransformListener e Buffer vêm quase sempre juntos – Buffer armazena os frames conhecidos até o momento e disponibiliza eles ao TransformListener
- Resto da construção da classe como de costume

```
class CargoTFNode : public rclcpp::Node{
public:
    CargoTFNode() : Node("cargo_tf2_pub")
    {
        this->declare_parameter<double>("cargo_distance", 0.5);
        cargo_dist_ = this->get_parameter("cargo_distance").as_double();
        this->declare_parameter<std::string>("drone_tf_name", "drone_frame");
        drone_frame_ = this->get_parameter("drone_tf_name").as_string();

        tf_broadcaster_ =
            std::make_unique<tf2_ros::TransformBroadcaster>(*this);
        tf_buffer_ =
            std::make_unique<tf2_ros::Buffer>(this->get_clock());
        tf_listener_ =
            std::make_shared<tf2_ros::TransformListener>(*tf_buffer_);

        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(50),
            std::bind(&CargoTFNode::timer_cb, this)
        );
    }

private:
    // Membros ROS
    std::unique_ptr<tf2_ros::TransformBroadcaster> tf_broadcaster_;
    std::shared_ptr<tf2_ros::TransformListener> tf_listener_{nullptr};
    std::unique_ptr<tf2_ros::Buffer> tf_buffer_;
    rclcpp::TimerBase::SharedPtr timer_;

    void timer_cb();

    double cargo_dist_ = 0.0;
    std::string drone_frame_ = "drone_frame";
};
```

CÓDIGO DO PACOTE TF2_DEMO

cargo_pubsub: timer_cb()

- Busca transformada mais recente do frame **D**rone com relação ao **M**apa: ${}^M T_D$
 - Se não estiver disponível, escreve um aviso e retorna o callback sem fazer nada
- Se estiver disponível, fazemos algumas contas:
 - Posição desejada do **C**argo: ${}^M \mathbf{p}_C = {}^M \mathbf{p}_D + {}^M \mathbf{p}_{D \rightarrow C}$
 - No caso ${}^M \mathbf{p}_{D \rightarrow C} = [0, 0, -0.5]^T$
 - Orientação desejada: ${}^M R_C = I$ (mesma orientação do mapa)
 - Com isso temos ${}^M T_C$; calculamos ${}^D T_M^{-1} = {}^D T_M$
 - Publicamos o frame **C** com relação a **D**: ${}^D T_C$
 - Sabendo ${}^D T_M$ e ${}^M T_C$, calculamos ${}^D T_C = {}^D T_M {}^M T_C$

Finalmente, escreve a mensagem e publica

```
void CargoTFNode::timer_cb(){
    std::string fromFrameRel = "map";
    std::string toFrameRel = drone_frame_;

    //Busca pelas TFs desejadas no cache
    geometry_msgs::msg::TransformStamped tf_map2drone;
    try {
        tf_map2drone = tf_buffer_>lookupTransform(
            fromFrameRel, toFrameRel,
            tf2::TimePointZero);
    } catch (const tf2::TransformException & ex) {
        auto& clk = *this->get_clock();
        RCLCPP_WARN_THROTTLE(
            this->get_logger(), clk, 1000,
            "Não foi encontrada TF de %s para %s: %s",
            toFrameRel.c_str(), fromFrameRel.c_str(), ex.what()
        );
        return;
    }

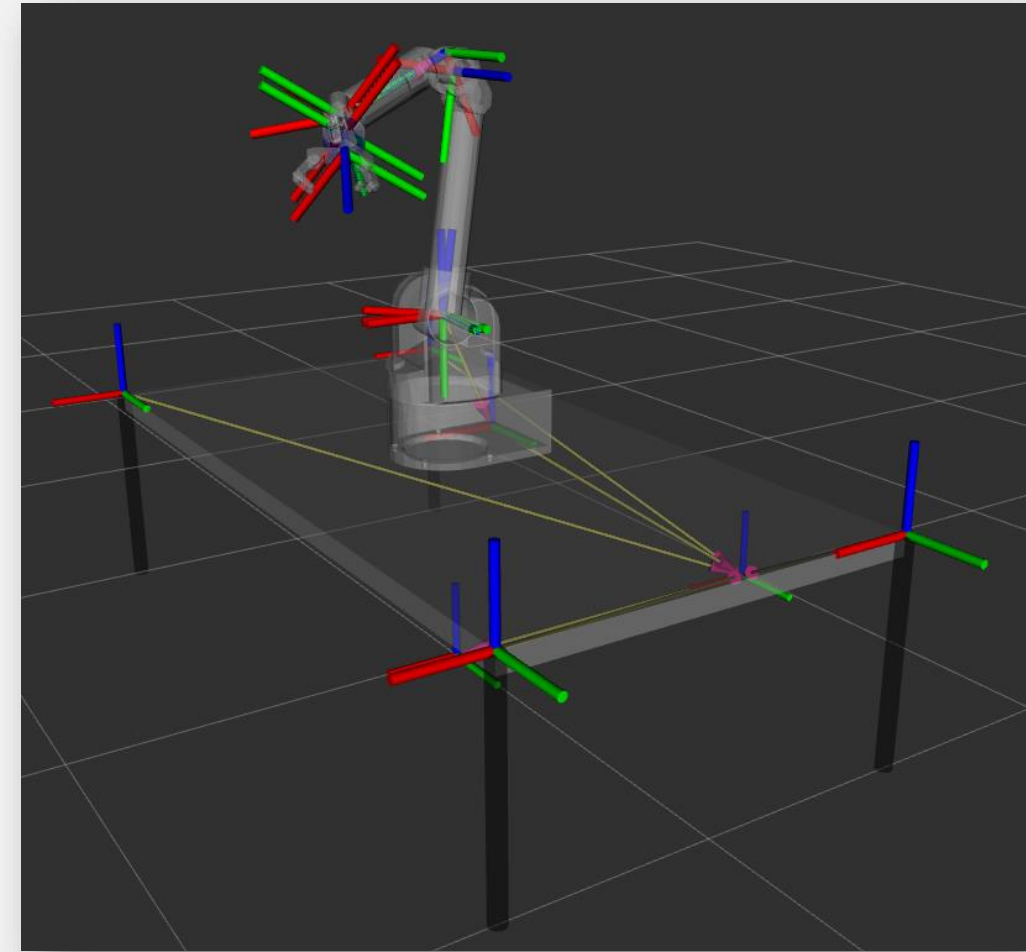
    //Cálculos de transformação
    tf2::Transform TF_M2D;
    tf2::fromMsg(tf_map2drone.transform, TF_M2D);
    tf2::Vector3 p_M2C = TF_M2D.getOrigin() + tf2::Vector3(0.0, 0.0, -0.5);
    tf2::Quaternion q_M2C(0, 0, 0, 1);
    tf2::Transform TF_M2C(q_M2C, p_M2C);
    tf2::Transform TF_D2C = TF_M2D.inverse() * TF_M2C;

    // Cria e preenche a mensagem de transformação
    geometry_msgs::msg::TransformStamped cargo_tf;
    cargo_tf.header.stamp = tf_map2drone.header.stamp;
    cargo_tf.header.frame_id = toFrameRel;
    cargo_tf.child_frame_id = "cargo_dynamic_frame";
    cargo_tf.transform = tf2::toMsg(TF_D2C);

    tf_broadcaster_>sendTransform(cargo_tf);
}
```

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- Arquivo tipo XML usado para descrever um robô
 - Estrutura : Colisões (hit boxes)
 - Cinemática : Juntas e links
 - Inércia : Massa, Centro de massa
- Utiliza sistema internacional [SI] de unidades
- Integração com pacotes de controle
- Interpretado por `robot_state_publisher`, `rviz`, `gazebo` e outros



Robô Barrett WAM sobre uma mesa, descritos com URDF.

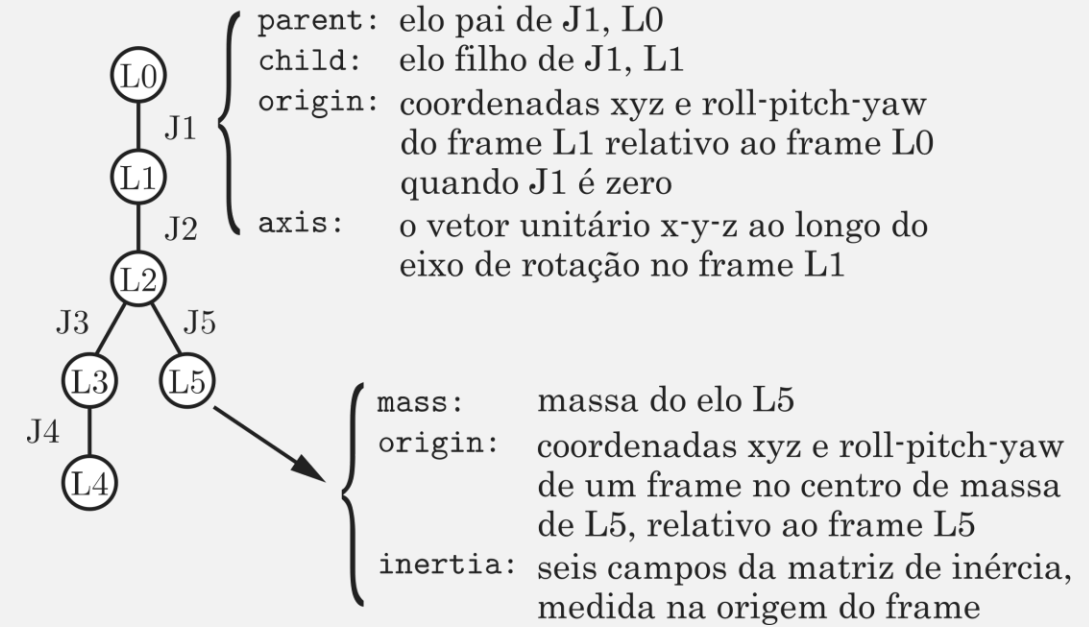
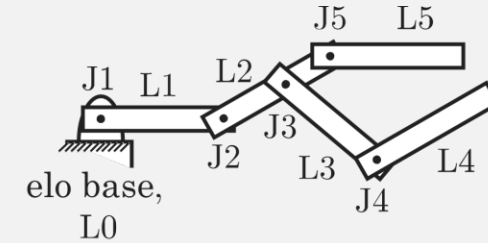
Tanto robô quanto mesa estão descritos em URDF, mas em arquivos diferentes. É possível combinar múltiplos arquivos URDF pelo linguagem de macros para XML [xacro](#).

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- Estrutura geral

```
<?xml version="1.0"?>
<robot name="meu_robo">
  <!-- links, joints, transmissions, etc. -->
</robot>
```

- Dentro de <robot> vão os elementos <link>, <joint>, <transmission> e opcionais como <gazebo> ou <material>



Robô de 5 elos descrito como uma árvore. Os nós da árvore são os elos, e as arestas são as juntas.

O URDF adota uma estrutura de árvore para representar os robôs.

URDF - LINK

Link : Define um corpo no espaço, sua aparência e propriedades físicas

Aparência : <visual>

<origin>

<geometry>

<material>

<color>

Colisão ("hitbox"): <collision>

<origin>

<geometry>

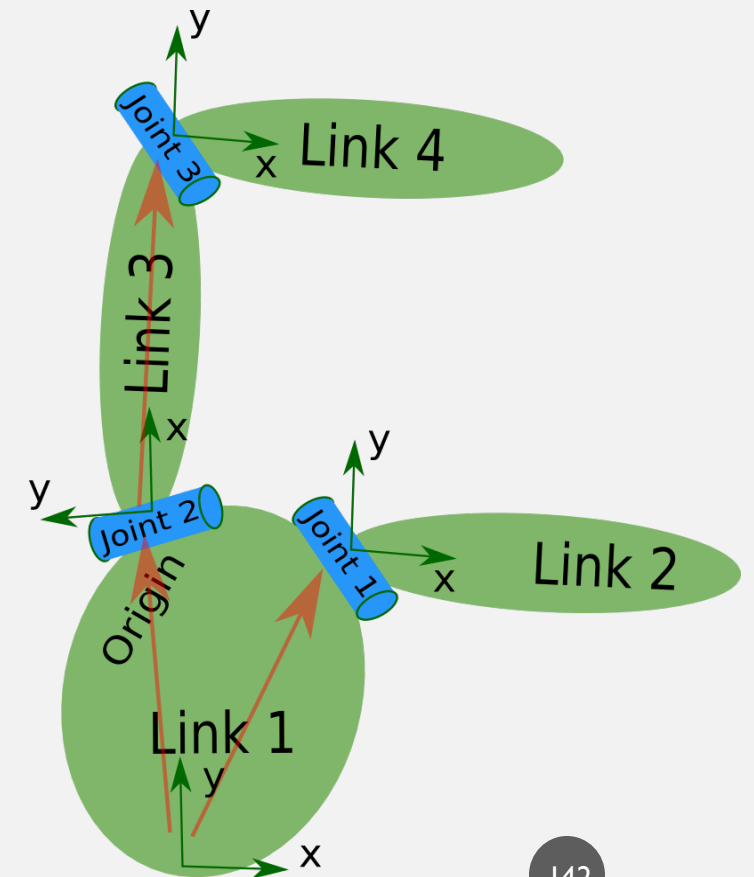
Física :

<inertia>

<origin>

<mass>

<inertia>



URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- **<origin>** : Define **posição e orientação** do elemento (por exemplo, onde o visual, colisão ou junta estão) **em relação ao frame pai**.
 - `<origin xyz="x y z" rpy="roll pitch yaw"/>`
 - Opcional : default = 0 0 0 | 0 0 0
- **<mass>** : Define a **massa total do link**.
 - `<mass value="1.5"/>`
- **<inertia>** : Define o **tensor de inércia**, ou seja, como a massa se distribui em torno do centro de massa.
 - `<inertia ixx="0.1" iyy="0.1" izz="0.1" ixy="0.0" ixz="0.0" iyz="0.0"/>`

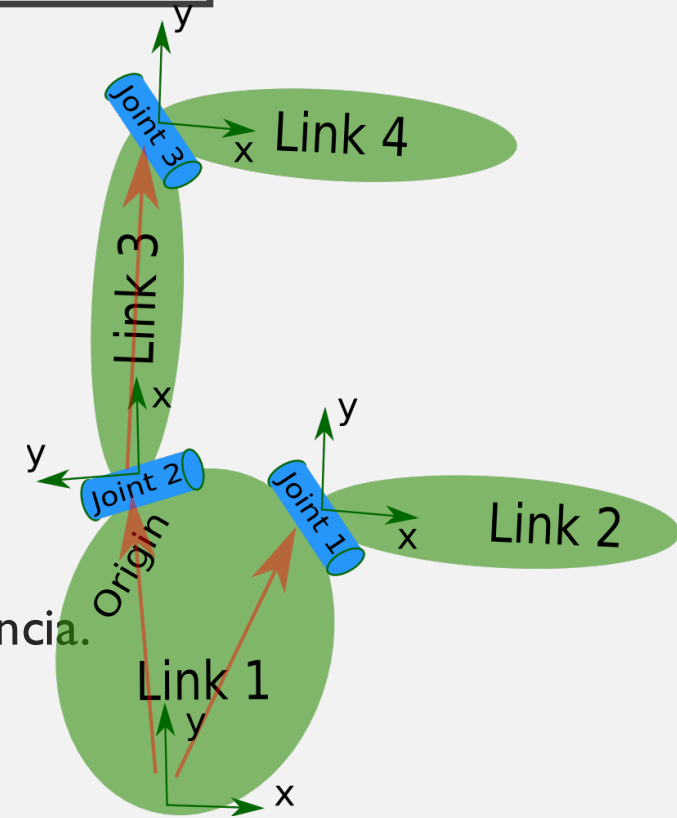
URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- **<geometry>**
 - Define a **forma geométrica** de um componente visual ou de colisão.

Nome	Sintaxe
box	<code><box size="x y z"/></code>
cylinder	<code><cylinder radius="r" length="l"/></code>
sphere	<code><sphere radius="r"/></code>

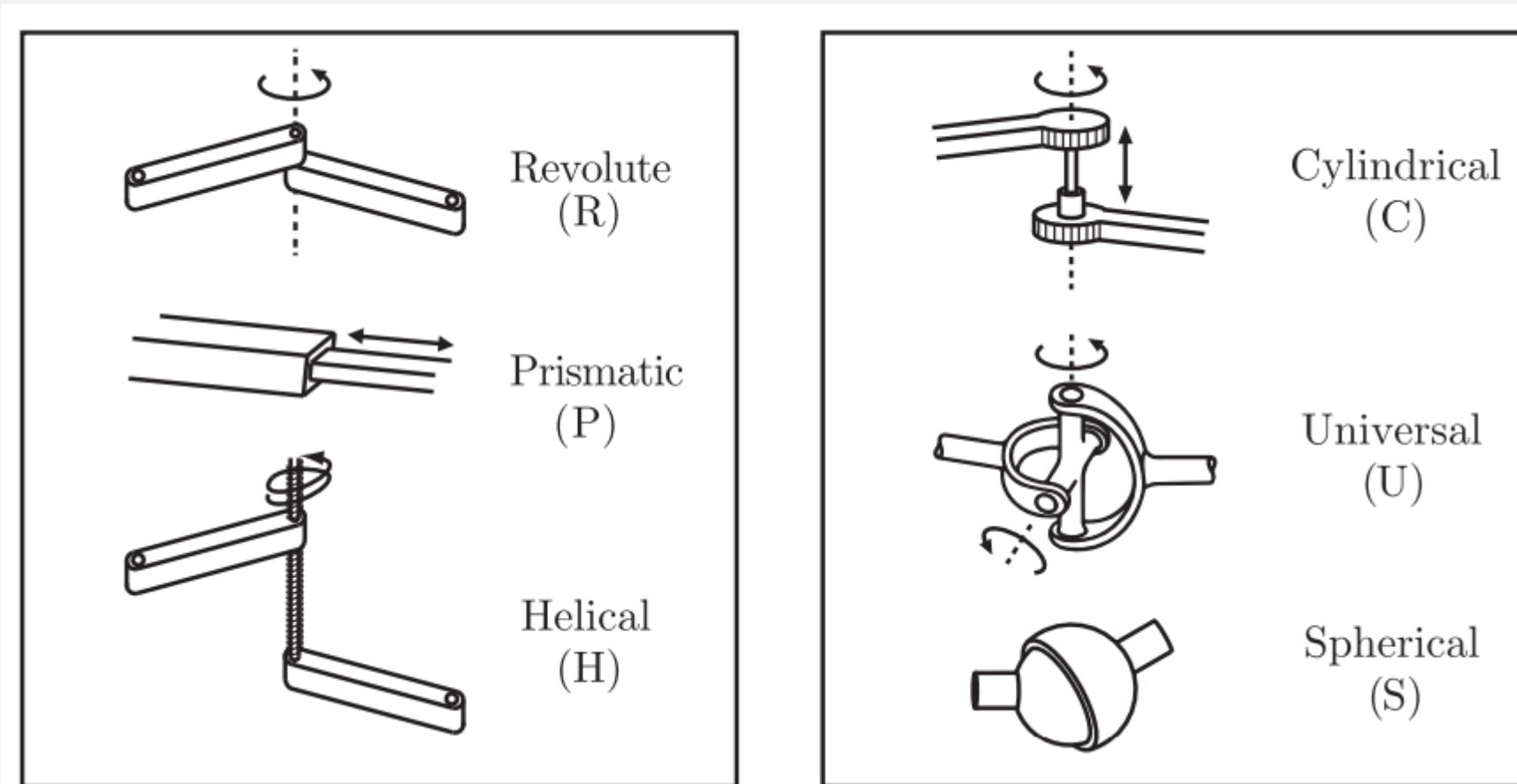
URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- Joint (elo) : Define a relação entre os links
 - **name**: Identificador do elo.
 - **type**: Tipo de relação (grau de liberdade entre pai e filho)
 - Fixed, revolute, continuous, prismatic, floating, planar.
 - **<parent>** e **<child>**: indicam quais links são conectados e qual a referência.
 - **<origin>**: posição e orientação do joint em relação ao link pai.
 - **<axis>**: eixo de rotação ou translação (somente para revolute, continuous, prismatic).
 - **<limit>**: restrições físicas e de força (obrigatório em revolute e prismatic).



URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- **Joint (elo)** : tipos de juntas físicas mais comuns



URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

Joint (elo) : types implementados em URDF

Tipo	Graus de liberdade	Movimento	Campos
fixed	0	Nenhum	<parent>, <child>
revolute	1	Rotação lim.	<axis>, <limit>
continuous	1	Rotação inf.	<axis>
prismatic	1	Linear	<axis>, <limit>
planar	3	Em um plano (2 trans + 1 rot)	<axis>
floating	6	Livre	-

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)



URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

Transmissions : Vincula um atuador ao joint (usado em simulação/controladores)

- Faz a ponte entre a *joint* (descrita no URDF) e o *actuator* (hardware interface)
 - Faz a joint que pode girar, girar
- Usado para integrar com controladores (ros2_control, Gazebo)
 - São as transmissões que permitem o ros2_control mapear comandos (effort, position, velocity) para o hardware.
- Indica a relação entre o movimento do atuador e da junta (ex: engrenagem, redução).

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

```
<transmission name="wheel_joint_trans">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="wheel_joint">
    <hardwareInterface>hardware_interface/VelocityJointInterface</
hardwareInterface>
  </joint>
  <actuator name="wheel_motor">
    <hardwareInterface>hardware_interface/VelocityJointInterface</
hardwareInterface>
    <mechanicalReduction>2.0</mechanicalReduction>
  </actuator>
</transmission>
```

- Um controlador envia comando para o atuador
- A transmissão aplica a redução e aplica na junta

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

- **Transmissions :**
 - Se não há transmission, o modelo move a junta “diretamente”, ignorando engrenagens ou reduções
 - Pode levar a erros de simulação, integração e controle

URDF (UNIFIED ROBOT DESCRIPTION FORMAT)

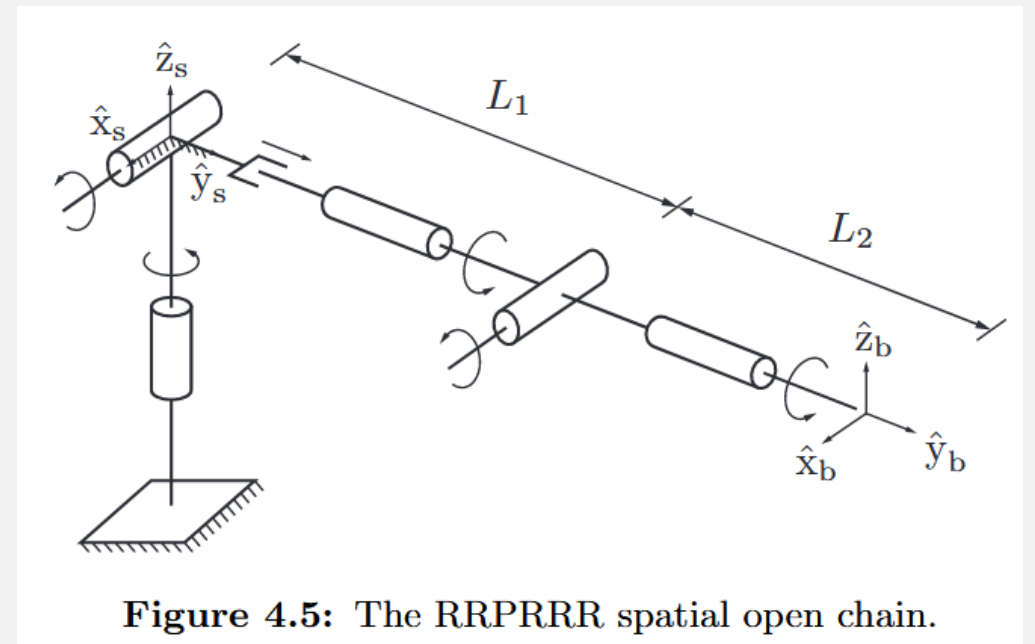
Hardware interfaces

- A hardware interface é o meio através do qual o controlador acessa ou comanda a junta ou o atuador.
Exemplo de tipos:
 - `hardware_interface/PositionJointInterface`
 - `hardware_interface/VelosityJointInterface`
 - `hardware_interface/EffortJointInterface`
- A definição de uma hardware interface permite que o framework `ros2_control` gere estruturas internas que indicam quais recursos de hardware estão disponíveis.

EXERCÍCIO PARA PRÓXIMA AULA

Faça, em URDF, o seguinte modelo de robô:

- Definição dos frames é de livre escolha
- Faça um pacote para fazer o launch do mesmo no RViz, junto com os nós `robot_state_publisher` e `joint_state_publisher_gui`
- Recomendado: use a extensão `xacro` para modularizar o seu código, fazer cálculos de inércia, etc...



Outras cadeias cinemáticas de inspiração... nem todas são fáceis de fazer no URDF!

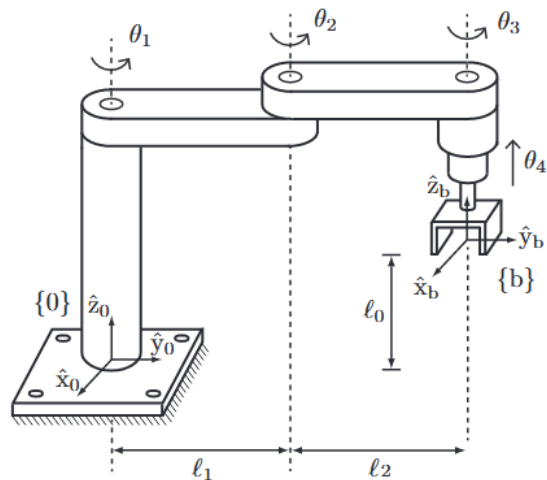


Figure 4.12: An RRRP SCARA robot for performing pick-and-place operations.

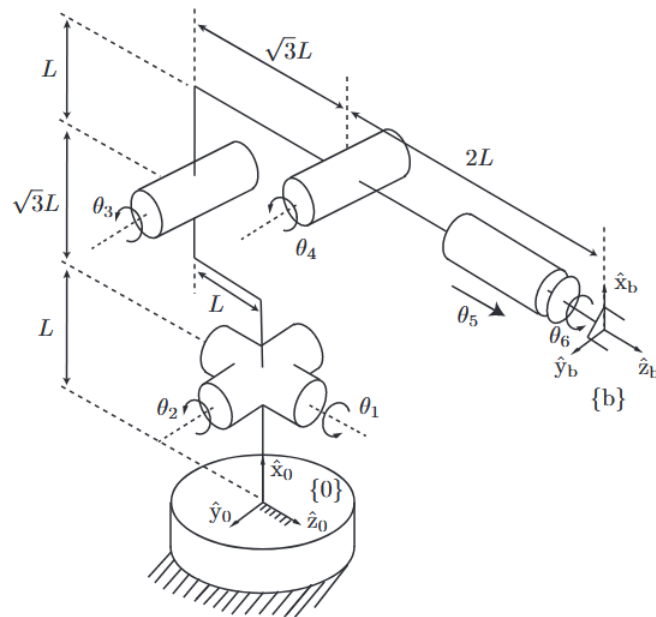


Figure 4.16: A URRPR spatial open-chain robot.

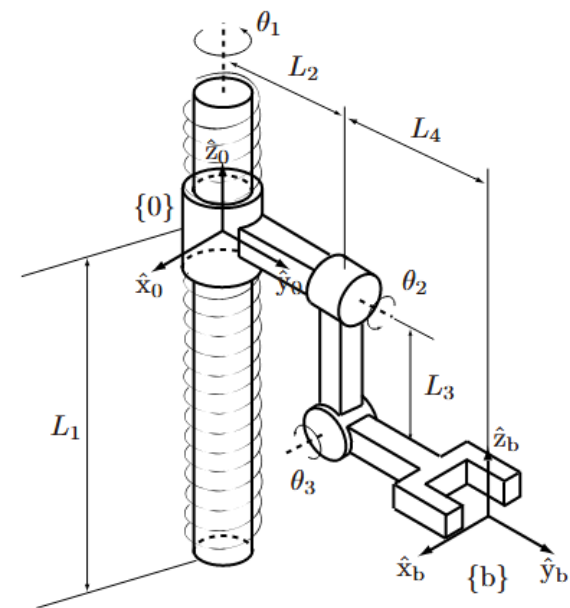


Figure 4.21: HRR robot. The pitch of the screw joint is denoted by h .

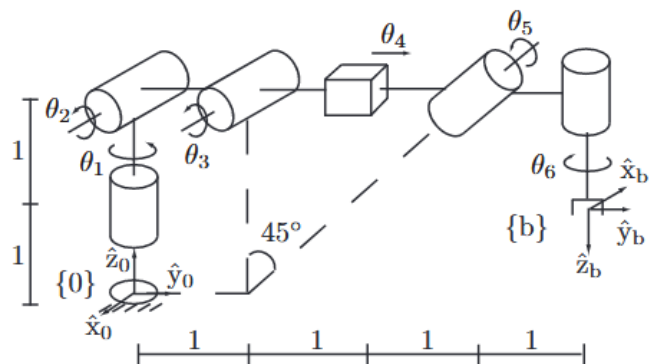
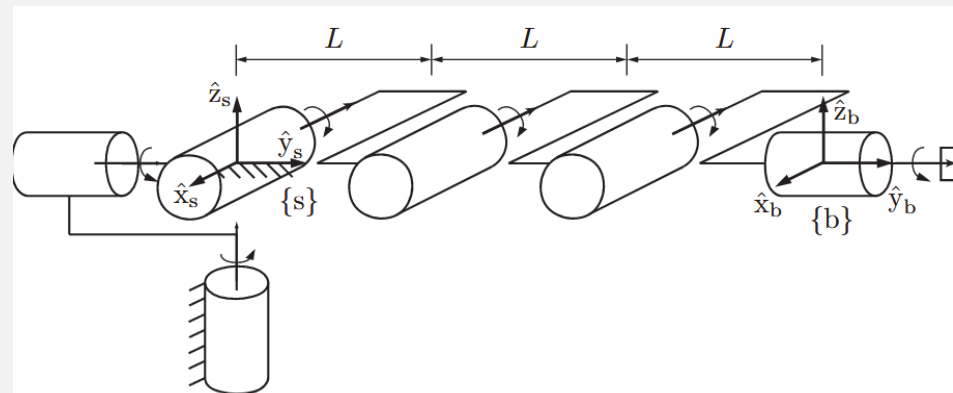


Figure 4.19: A spatial RRRPRR open chain with prescribed fixed and end-effector frames.



Cadeia 6R – estrutura cinemática usada na maior parte de manipuladores industriais